AI usage log

Tools used

Tool	Used for
Claude Code (Claude Opus 5.5, in VS Code)	Data exploration, writing all the code, the back-test, and first drafts of the memo and these logs. I directed the work, checked the numbers, and made the calls on rules and trade-offs.
Claude API (`claude-opus-5-5`, optional at runtime)	Inside the product: judging addresses and names, and reading free-text WhatsApp replies. Not used for the memo numbers: no API key was set during the build, so those use the rule-based fallback.

How I kept the AI honest

- Every number in the memo comes from `scripts/analyse.py` → `reports/findings.md`, never from the AI's prose. If it isn't in the generated report, it isn't in the memo.
- Each rule was judged by its ₹ effect in a back-test on real outcomes, in two separate periods, not by whether it sounded sensible.
- Screenshots of the dashboard and simulator in a real (headless) browser, and 25 unit tests on the ₹ maths, rules and WhatsApp timers.

Where the AI got it wrong or weak, and how it was caught

1. Rules that "obviously" cut RTO actually lost money. The AI's first rule set held any order with an invalid phone and any "possible duplicate" (same phone and items within 24h). Both sound like textbook RTO prevention. *Caught by:* breaking the back-test down rule by rule. The duplicate rule lost **₹7,296** on Apr–Aug orders: the pairs were real double-submits, but both copies were usually delivered. The invalid-phone rule lost ₹485 in Apr–Aug and ₹980 in September, because those orders deliver about 50% of the time. *Fix:* duplicates go to WhatsApp confirm, and invalid phones ship and are flagged to the courier. This is the most useful lesson of the project: plausible-sounding rules need a ₹ back-test.

2. Thresholds picked by intuition were fragile. The AI's first thresholds (nudge 25% / confirm 35% / hold 75%) looked reasonable and were positive in the base case. *Caught by:* asking "what if WhatsApp confirmation works worse than we assumed?". Under weaker assumptions (catches 25% of RTOs, loses 15% of genuine buyers) that set went **negative (–₹700 in September)**. *Fix:* compared candidates under both assumptions and chose 25 / 50 / 75, which stays at +₹1,737 in the weak case. The robustness table is in the findings.

3. The address checker mislabelled a third of good addresses. The AI's regex for house numbers didn't match "House No. 380" (the "No." broke it), so about 350 complete addresses were labelled "landmark only". *Caught by:* printing which address patterns landed in each bucket. Complete addresses showed up as 967 instead of ~1,320. *Fix:* corrected the regex and added a unit test with that exact address.

4. A WhatsApp customer could be cancelled with no final warning. The AI's first flow sent the "last call" at +20h only outside quiet hours. For a night order, +20h fell at 4am, so it was skipped and the order was cancelled at 08:00 without warning. *Caught by:* running a 02:10 order through the simulator and reading the transcript. *Fix:* the last call goes out in the morning, the deadline moves with it, and a test checks that no message is sent at night and the last call is never skipped.

5. Smaller ones:

- Status labels `RT0` (trailing space) and `Rto` first appeared as separate 0%-RTO groups in exploration. Caught by eyeballing value counts; fixed by normalising in the loader.
- Offline reply parsing didn't understand "haan bhej do", and "H.No 12, Shastri Nagar" was read as a "no" (because of "No"). Caught in the simulator; fixed by checking for an address before yes/no, and added to the tests.
- The first ₹ accounting could show *negative* "₹ lost" on nudges, because recovered RTOs were counted in the wrong column. Caught by reading the comparison table; rewritten so saved/lost are split by how each order would have ended, with a test that the expected figure equals the probability-weighted mix of both outcomes.
- The analysis header said the data started "18 Jun" because it printed the first row's date, not the earliest date. Caught by reading the report.